NAME – KUNAL RUSHIKESH PATIL

PRN – 202501100103

DIV – SE1

BATCH – S15

DATASET NAME – AMAZON PRODUCT DATASET

- The Amazon Product Dataset contains information such as product name, price, rating, stock, and discount. This dataset is used to analyze product performance and customer preferences. **NumPy** is used for numerical operations like calculating mean, sum, and sorting values efficiently. **Pandas** is used for data manipulation, such as filtering, grouping, and modifying the dataset. By using both libraries, we can easily analyze and extract useful insights from the data. This helps in better decision-making in e-commerce applications.

Got it — but just saying "apply 10 NumPy and 10 Pandas operations" isn't enough for a proper assignment. You need a **clear dataset + meaningful analysis**, otherwise it looks like random code.

Here's a **complete, structured assignment solution** you can directly use (with explanation + code).

- **Amazon Product Dataset Analysis using NumPy & Pandas**

Step 1: Import Libraries

```
import numpy as np
import pandas as pd
```

Step 2: Create Sample Amazon Dataset

```
data = {
   'Product_ID': [101, 102, 103, 104, 105],
   'Product_Name': ['Laptop', 'Mobile', 'Headphones', 'Tablet', 'Smartwatch'],
   'Price': [55000, 20000, 2000, 15000, 8000],
   'Rating': [4.5, 4.2, 4.0, 3.8, 4.3],
   'Stock': [10, 25, 50, 15, 20],
   'Discount (%)': [10, 5, 15, 8, 12]
}

df = pd.DataFrame(data)
print(df)

)
```

- **NUMPY OPERATIONS (10 Methods)**
- **Convert columns to NumPy arrays**

```
price = np.array(df['Price'])
rating = np.array(df['Rating'])
stock = np.array(df['Stock'])
```

Given:
Price = [55000, 20000, 2000, 15000, 8000]
Stock = [10, 25, 50, 15, 20]

1. Mean Price

```
print(np.mean(price))
```

OUTPUT :

20000.0

2. Median Price

```
print(np.median(price))
```

OUTPUT:

15000.0

3. Standard Deviation

print(np.std(price))

OUTPUT:

≈ 18248.29

4. Maximum Price

print(np.max(price))

OUTPUT:

55000

5. Minimum Price

print(np.min(price))

OUTPUT:

2000

6. Total Stock

print(np.sum(stock))

OUTPUT:

120

7. Square of Prices

print(np.square(price))

OUTPUT:

[3025000000 400000000 4000000 225000000 64000000]

8. Log of Prices

print(np.log(price))

OUTPUT:

[10.915 9.903 7.601 9.616 8.987] (approx)

9. Sort Prices

print(np.sort(price))

OUTPUT:

[2000 8000 15000 20000 55000]

10. Index of Maximum Price

print(np.argmax(price))

OUTPUT:

0

- **PANDAS OPERATIONS (10 Methods)**

  Given:

	Product_ID	Product_Name	Price	Rating	Stock	Discount (%)
0	101	Laptop	55000	4.5	10	10
1	102	Mobile	20000	4.2	25	5
2	103	Headphones	2000	4.0	50	15
3	104	Tablet	15000	3.8	15	8
4	105	Smartwatch	8000	4.3	20	12

1. Display first rows

print(df.head())

OUTPUT:

(Same as full dataset since only 5 rows)

2. Dataset Info

print(df.info())

OUTPUT:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 5 entries, 0 to 4

Columns: 6 entries

Data types: int64, float64, object

3. Statistical Summary

print(df.describe())

OUTPUT:

	Product_ID	Price	Rating	Stock	Discount (%)
count	5.000000	5.0	5.0000	5.000000	5.000000
mean	103.000000	20000.0	4.1600	24.000000	10.000000
std	1.581139	18248.3	0.2588	15.165751	3.807887
min	101.000000	2000.0	3.8000	10.000000	5.000000
max	105.000000	55000.0	4.5000	50.000000	15.000000

4. Sort by Price

print(df.sort_values(by='Price'))

OUTPUT:

Headphones 2000

Smartwatch 8000

Tablet 15000

Mobile 20000

Laptop 55000

5. Filter products with price > 10000

print(df[df['Price'] > 10000])

OUTPUT:

Laptop, Mobile, Tablet

6. Add New Column (Final Price after Discount)

df['Final Price'] = df['Price'] - (df['Price'] * df['Discount (%)'] / 100)
print(df)

OUTPUT:

Laptop → 49500

Mobile → 19000

Headphones → 1700

Tablet → 13800

Smartwatch → 7040

7. Group by Rating (example)

print(df.groupby('Rating')['Price'].mean())

OUTPUT:

3.8 → 15000

4.0 → 2000

4.2 → 20000

4.3 → 8000

4.5 → 55000

8. Value Counts (if categorical)

print(df['Product_Name'].value_counts())

OUTPUT:

Each product appears once → count = 1

9. Check Null Values

print(df.isnull().sum())

OUTPUT:

All columns → 0

10. Drop a Column

df = df.drop(columns=['Stock'])
print(df)

OUTPUT:

Columns left:

Product_ID, Product_Name, Price, Rating, Discount (%), Final Price